Tutorial de construcción de una PWA local con Vite y React

T4 Actividad 2
Formulario de usuarios Login Operaciones CRUD

Este tutorial explica cómo se construyó React Lab y cómo ejecutar el proyecto en la computadora. Relaciona los archivos con las notas 4.6 a 4.13 y 6.1 a 6.6, conservando la numeración de los PDF entregados.

El resultado es una aplicación con formularios, navegación por módulos, consulta de una API y catálogos con alta, consulta, modificación y baja. Incluye dos formas de guardar datos: almacenamiento del navegador y un servidor Express local. No requiere publicar un sitio.

1 Preparar el proyecto

La base corresponde a la nota 6.1. Se utilizó Vite para preparar y compilar una interfaz React escrita en JavaScript. Node.js ejecuta las herramientas; JavaScript es el lenguaje, mientras que Express implementa la API del servidor.

1. Abrir la carpeta C:\LSW\ACT\t4-actividad-2 en Visual Studio Code. Verificar Node.js y npm desde una terminal con node --version y npm --version. El proyecto se compiló con Node 26.

2. Para trabajar con el proyecto entregado, instalar exactamente las dependencias del archivo package-lock.json y ejecutar Vite:

```
cd C:\LSW\ACT\t4-actividad-2
npm ci
npm run dev
```

Abrir la dirección Local que imprime Vite y mantener la terminal en ejecución. Para detenerla se utiliza Ctrl+C. El puerto puede variar si el habitual está ocupado.

3. Si se quiere practicar desde cero en otra carpeta vacía, el comando equivalente de inicio es:

```
npm create vite@latest react-lab-practica -- --template react
cd react-lab-practica
npm install
```

Después se incorporan los archivos y dependencias descritos en este tutorial. No se debe ejecutar este comando dentro de la carpeta del proyecto ya terminado.

2 Instalar y organizar las dependencias

Notas 6.1 y 6.2

Las dependencias separan la interfaz, el estado y el acceso a datos. package.json declara los paquetes y comandos; package-lock.json conserva las versiones instaladas para reproducir el entorno.

Paquetes	Uso en la aplicación
react y react-dom	Componentes, hooks y renderizado de la interfaz.
vite y @vitejs/plugin-react	Servidor de desarrollo y compilación.
react-router-dom	Rutas, navegación y vistas anidadas.
@mui/material y @mui/icons-material	AppBar, tablas, diálogos, botones y pestañas.
@emotion/react y @emotion/styled	Motor de estilos de Material UI.
@reduxjs/toolkit y react-redux	Estado global del catálogo de institutos.
axios y express	Cliente HTTP y API REST local.
vite-plugin-pwa	Manifest y service worker de la PWA.

Comandos para una instalación desde cero

```
npm install react-router-dom axios express
npm install @mui/material @mui/icons-material
npm install @emotion/react @emotion/styled
npm install @reduxjs/toolkit react-redux
npm install -D vite-plugin-pwa
```

Estructura principal

```
t4-actividad-2/
  public/                Iconos de la aplicación
  src/
    api/                 Repositorio y cliente Axios
    components/          Layout, formularios y tabla CRUD
    domain/              Esquemas, datos iniciales y reglas
    hooks/               useForm, useCounter, useUsuarios
    pages/               Inicio, login, catálogos y laboratorio
    security/            Autenticación, institutos y Redux
    share/css/           Estilos generales
    main.jsx             Punto de entrada de React
    AppAllModules.jsx    Configuración de las rutas
  server/                API Express y persistencia
  tests/                 Pruebas de reglas y endpoints
```

main.jsx monta AppAllModules en el elemento root de index.html. ThemeProvider aplica el tema de Material UI, Provider entrega el store de Redux y AuthProvider comparte la sesión entre las páginas.

3 Implementar los primeros hooks

Notas 4.6 y 4.7

Un hook permite utilizar estado y otras funciones de React dentro de un componente. El primer ejercicio fue un contador: al cambiar su estado, React vuelve a renderizar el valor mostrado en pantalla.

Contador con useState

1. En src/pages/HooksLab.jsx se declara un estado inicial de cero y la función que lo actualiza.

```
const [count, setCount] = useState(0);

// Sumar y restar a partir del estado anterior.
setCount(c => c + 1);
setCount(c => c - 1);

// Reiniciar.
setCount(0);
```

2. Los botones llaman estas funciones mediante onClick. La actualización funcional, como c => c + 1, calcula el siguiente valor a partir del anterior.

En la vista Hooks de React, el primer contador comienza en cero. Sumar, restar y reiniciar permiten observar el estado sin consultar la consola.

Extraer la lógica a useCounter

3. Para reutilizar el contador se creó src/hooks/useCounter.js. El hook recibe el valor inicial y expone el estado y las operaciones.

```
import { useState } from 'react';

export function useCounter(initial = 0) {
  const [valor, setValor] = useState(initial);
  return {
    valor,
    acumular: amount => setValor(v => v + amount),
    reset: () => setValor(initial)
  };
}
```

4. El componente utiliza useCounter(10), por lo que el segundo contador inicia en diez. acumular(1) incrementa el valor y acumular(-1) lo reduce; reset lo devuelve a diez.

Resultado esperado

Cada contador cambia de manera independiente. useState administra el estado en el componente, mientras que useCounter encapsula la misma lógica en una función reutilizable cuyo nombre comienza con use.

Esta separación se aplica después a los formularios y los catálogos: la vista describe lo que se muestra y el hook organiza las operaciones que modifican sus datos.

4 Construir el formulario y el login

Notas 4.8, 4.12 y 4.13

El login combina un formulario controlado con un reducer. El formulario guarda lo escrito en correo y contraseña; el reducer representa la etapa de la sesión. Los usuarios del catálogo son registros de demostración y no se convierten automáticamente en cuentas de acceso.

Crear el hook useForm

1. En src/hooks/useForm.js se guarda el objeto del formulario y se modifica únicamente el campo que cambia. La versión siguiente conserva la lógica del archivo, con líneas separadas para facilitar la lectura.

```
export function useForm(initialState = {}) {
  const [formulario, setFormulario] = useState(initialState);
  const onChange = (value, campo) =>
    setFormulario(previous => ({
      ...previous, [campo]: value
    }));
  const reset = (next = initialState) => setFormulario(next);
  return { ...formulario, formulario, onChange, reset };
}
```

2. En src/pages/Login.jsx se inicializan email y password. Cada TextField recibe su value y ejecuta onChange con el nombre del campo. El evento submit evita la recarga, comprueba los datos y llama a auth.login.

```
const { formulario, onChange, reset } = useForm({
  email: '', password: ''
});
// Dentro del campo de correo:
value={formulario.email}
onChange={e => onChange(e.target.value, 'email')}
```

Controlar la sesión con useReducer

3. src/security/auth/AuthContext.jsx define authReducer y AuthProvider. El reducer recibe acciones y devuelve un nuevo estado, que contiene status, user y error.

Acción	Estado resultante	Significado
checking	checking	Se está validando el acceso.
login	authenticated	Se muestra la sesión del usuario.
error	guest	Se informa el error y no se permite entrar.
logout	guest	Se cierra la sesión.

El acceso de prueba es demo@reactlab.test con la contraseña React123!. El botón Usar credenciales demo llena los campos; Entrar al laboratorio envía el formulario. En modo local la comparación ocurre en el navegador. En modo API se envía POST /login al servidor.

Protected redirige al login cuando se intenta abrir un catálogo sin sesión. Al recargar, el contexto inicia como invitado y se solicita entrar nuevamente. Los datos guardados del catálogo permanecen.

5 Consultar usuarios y paginar la API

Notas 4.9, 4.10 y 4.11

La vista Consulta de API utiliza ReqRes para practicar peticiones HTTP GET. Es independiente del catálogo Usuarios, que permite guardar cambios. src/hooks/useUsuarios.js concentra la consulta y src/pages/RemoteUsers.jsx presenta los resultados.

1. Crear una instancia de Axios con la URL base y un tiempo máximo de espera.

```
const reqRespApi = axios.create({
  baseURL: 'https://reqres.in/api',
  timeout: 12000
});
```

2. Declarar estados para usuarios, página, total de páginas, carga, error y reintento. useEffect consulta la API cuando cambian page o retry. El siguiente fragmento muestra la parte central de la petición.

```
reqRespApi.get('/users', {
  params: { page },
  signal: controller.signal
}).then(({ data }) => {
  if (!Array.isArray(data.data)) {
    throw new Error('Formato de respuesta inesperado.');
  }
  setUsuarios(data.data);
  setTotalPages(data.total_pages || 1);
  ficRefPage.current = page;
});
```

3. El estado page controla qué página se consulta y provoca un nuevo render. La referencia ficRefPage conserva el último número cargado sin producir un render adicional; en esta implementación no dirige la paginación.

4. Los botones Anterior y Siguiente cambian page dentro de los límites. Se deshabilitan durante la carga y al alcanzar los extremos; Siguiente también se deshabilita ante un error.

```
previous: () => setPage(p => Math.max(1, p - 1)),
next: () => setPage(p => Math.min(totalPages, p + 1))
```

Manejo de carga y errores

Mientras se espera, se muestra una barra de progreso. Si la petición falla se presenta un mensaje y un botón Reintentar. AbortController cancela la solicitud al desmontar el componente o cambiar de página, evitando procesar respuestas que ya no corresponden a la vista.

Para probarlo, abrir Consulta de API con conexión a Internet y revisar avatar, nombre, correo e ID. Cambiar de página y comprobar que los datos se actualizan. Esta consulta depende de la disponibilidad y de los límites del servicio externo.

ReqRes sirve aquí como ejemplo de lectura. Las altas, modificaciones y bajas persistentes de la actividad se realizan con el repositorio propio, no con las escrituras de demostración de ReqRes.

6 Desarrollar el CRUD de los catálogos

CRUD reúne cuatro operaciones: Create, Read, Update y Delete. En la interfaz corresponden a Nuevo, Consultar, Editar y Eliminar. La tabla se reutiliza en Usuarios y en los catálogos de los módulos.

1. Definir en src/domain/catalogs.js los campos de cada entidad, su etiqueta, tipo y opciones. Por ejemplo, users contiene name, email y role; los roles disponibles son Alumno, Docente y Administrador.

2. Crear en src/domain/seed.js los registros iniciales. Estos datos permiten comenzar la práctica sin llenar todos los catálogos manualmente.

3. Implementar saveToDatabase y removeFromDatabase en src/domain/database.js. saveToDatabase valida, normaliza y agrega o reemplaza un registro. removeFromDatabase elimina por ID y atiende las relaciones entre institutos y negocios.

4. Separar la persistencia en src/api/repository.js. Sus tres métodos conservan la misma interfaz aunque cambie el almacenamiento.

Operación	Método del repositorio	Comportamiento
Consultar	list(entity)	Devuelve los registros del catálogo.
Crear o editar	save(entity, input)	Sin ID crea; con ID modifica.
Eliminar	remove(entity, id)	Elimina el registro seleccionado.

5. Conectar el repositorio mediante src/hooks/useCatalog.js. El hook expone rows, loading, error, reload, save y remove. Después de guardar o eliminar vuelve a cargar el listado.

6. Renderizar src/components/CrudTable.jsx. El componente muestra la tabla, búsqueda, paginación, detalle y los diálogos. RecordForm utiliza useForm para editar los campos del esquema.

Reglas que se verifican antes de guardar

Se comprueban campos obligatorios, longitud máxima de 240 caracteres, formato de correo, opciones de listas, números no negativos y URL HTTP o HTTPS. Se rechazan correos duplicados en Usuarios y claves duplicadas en Institutos, Productos y Carreras. Los periodos comprueban que la fecha final no preceda a la inicial.

Ejemplo de recorrido

En Usuarios, pulsar Nuevo usuario, capturar un nombre, correo y rol, y guardar. Buscar el nombre, consultar su detalle y editar el rol. La eliminación abre una confirmación. Un mensaje informa cuando la operación termina correctamente; si falla, el diálogo conserva los datos para corregirlos.

En modo local se guarda un objeto JSON bajo la clave t4-react-lab-v1 de localStorage. El almacenamiento depende del origen: cambiar de puerto, de navegador o de localhost a 127.0.0.1 puede mostrar otro conjunto de datos. Para comprobar persistencia debe usarse siempre la misma dirección.

7 Administrar institutos con Redux

Nota 6.5

El catálogo de Institutos utiliza Redux Toolkit para centralizar su estado. La tabla sigue siendo CrudTable, pero los datos y las operaciones provienen del store. Así se comparan dos formas de manejar el estado: custom hooks para catálogos y Redux para institutos.

1. Crear src/security/redux/actions/institutesActions.js con getInstitutesAll, saveInstitute y deleteInstitute. Estas funciones llaman al mismo repositorio usado por el resto de la aplicación.

2. Crear thunks.js. createAsyncThunk representa una operación asíncrona y emite eventos pending, fulfilled y rejected.

```
export const GET_DATA_START = createAsyncThunk(
  'institutes/load', getInstitutesAll
);
export const SAVE_INSTITUTE = createAsyncThunk(
  'institutes/save', saveInstitute
);
export const DELETE_INSTITUTE = createAsyncThunk(
  'institutes/delete',
  async id => { await deleteInstitute(id); return id; }
);
```

3. Crear slices/institutesSlice.js. Su estado contiene institutesDataArr, loading y error. El listado se sustituye al cargar; al guardar se añade o reemplaza un registro; al eliminar se filtra el ID.

4. Crear store/store.js y registrar institutesReducer.

```
import { configureStore } from '@reduxjs/toolkit';
import institutesReducer from '../slices/institutesSlice.js';

export default configureStore({
  reducer: { institutesReducer }
});
```

5. En main.jsx, envolver la aplicación con Provider y pasar store. En Institutes.jsx, useSelector lee institutesReducer y useDispatch permite ejecutar los thunks.

```
const state = useSelector(s => s.institutesReducer);
useEffect(() => {
  dispatch(GET_DATA_START());
}, [dispatch]);

// La tabla espera una promesa que pueda fallar.
save: r => dispatch(SAVE_INSTITUTE(r)).unwrap()
```

unwrap permite que el formulario capture un error del thunk y muestre el mensaje correspondiente. El estado de Redux vive en memoria; la persistencia se consigue porque las acciones utilizan repository, que guarda en localStorage o en la API según la configuración.

8 Crear rutas y pestañas relacionadas

Notas 6.3, 6.4 y 6.6

Navegación entre módulos

AppAllModules.jsx define un router con createHashRouter. Las rutas contienen #, por ejemplo /#/security/users. Layout presenta el menú lateral, la AppBar y un Outlet donde se carga la vista seleccionada. ModulePage incorpora otro Outlet para los catálogos de cada módulo.

Módulo	Rutas principales
Seguridad	/security/users, /security/institutes, /security/labels, /security/periods
Educación	/education/courses, /education/groups
Comercio	/ecommerce/products, /prices, /orders, /payments, /shippings, /inventories
Laboratorio	/lab/hooks y /lab/api

En Comercio, los nombres abreviados de la tabla comparten el prefijo /ecommerce. Cada vista implementa el mismo CRUD de catálogos; no constituye un sistema completo de ventas, facturación o control escolar.

AppBar y diseño adaptable

src/components/Layout.jsx usa AppBar y Toolbar de Material UI. Muestra el modo de almacenamiento, acceso a la sesión y controles de instalación. NavLink marca la opción activa. En pantallas pequeñas, Drawer presenta el menú y el CSS ajusta tarjetas, formularios y tablas.

Relación entre institutos y negocios

1. En src/security/institutes/pages/Institutes.jsx se crearon las pestañas Institutos y Negocios. El estado instituteId guarda el instituto seleccionado y habilita Negocios.

2. Cada negocio guarda parentId con el ID del instituto. El listado filtra por este campo y muestra únicamente sus negocios.

3. Después de seleccionar un negocio se habilitan Info adicional, Archivos, Teléfonos, Direcciones web y Domicilios. Estos registros usan parentId con el ID del negocio.

```
Instituto
  Negocio A
    Información adicional
    Referencias de archivos
    Teléfonos y direcciones
  Negocio B
    Sus propios registros de detalle
```

La pestaña Archivos administra nombre y enlace a un archivo; no carga archivos binarios. Al eliminar un instituto se eliminan sus negocios y detalles; al eliminar un negocio se eliminan sus detalles. El diálogo advierte esta consecuencia antes de confirmar.

9 Implementar la API Express local

La segunda arquitectura mantiene la interfaz React y cambia la persistencia a un servidor en la misma computadora. La API utiliza Express y un archivo JSON; no requiere instalar un motor de base de datos.

Separación de responsabilidades

server/app.js define createApp, el middleware, la sesión y las rutas HTTP. server/index.js prepara la carpeta de datos, crea los registros iniciales si no existe el archivo y pone el servidor a escuchar. Esta separación permite probar las rutas con una base en memoria.

```
// Fragmento de server/index.js
const write = db => {
  writeFileSync(`${file}.tmp`, JSON.stringify(db, null, 2));
  renameSync(`${file}.tmp`, file);
};
if (!existsSync(file)) write(createSeed());
const read = () => JSON.parse(readFileSync(file, 'utf8'));

createApp({ read, write }).listen(port, '127.0.0.1');
```

file corresponde a server/data/database.json. La escritura usa un archivo temporal y luego lo renombra. El servidor escucha en 127.0.0.1:3001 por defecto, por lo que se utiliza desde la computadora local.

Rutas del recurso Institutos

Método	Ruta después de /api/v1/security	Resultado
POST	/login	Entrega token y usuario demo.
POST	/logout	Invalida la sesión.
GET	/institutos	Consulta el listado.
GET	/institutos/:id	Consulta un registro.
POST	/institutos	Crea un registro.
PUT	/institutos/:id	Actualiza un registro.
DELETE	/institutos/:id	Elimina un registro.

El frontend llama a la entidad institutes; repository la convierte en la ruta institutos. Los demás catálogos, como users, products y businesses, utilizan su nombre como ruta. La validación y las relaciones se reutilizan desde src/domain/database.js.

POST /login comprueba las credenciales de demostración y entrega un token con vigencia de una hora. Las operaciones del catálogo requieren Authorization: Bearer seguido del token. Las sesiones están en memoria: reiniciar la API las invalida. Este mecanismo está destinado al ejercicio local con datos de prueba.

10 Conectar React con la API local

1. Desde la raíz del proyecto, copiar el ejemplo de configuración si todavía no existe .env.local. No sobrescribir una configuración propia sin revisarla.

```
Copy-Item .env.example .env.local
```

2. Editar .env.local con los siguientes valores. Vite lee las variables que comienzan con VITE_ al iniciar o compilar la aplicación.

```
VITE_DATA_MODE=api
VITE_REST_API_SECURITY_EDUCATION=/api/v1/security
```

3. Abrir una primera terminal en la raíz del proyecto para ejecutar Express. Mantenerla abierta.

```
npm run api
```

4. Abrir una segunda terminal en la misma carpeta para iniciar React. Si Vite estaba ejecutándose antes de editar .env.local, detenerlo y volverlo a iniciar.

```
npm run dev
```

vite.config.js contiene un proxy: las solicitudes que empiezan con /api se reenvían a http://127.0.0.1:3001. Por ello, el navegador puede usar una ruta relativa sin configurar CORS para este recorrido local.

5. Entrar nuevamente con demo@reactlab.test y React123!. Confirmar que la barra superior muestra API Express. Crear o editar un instituto y revisar que server/data/database.json conserve el cambio.

Comprobar la API con Postman

Enviar POST a http://127.0.0.1:3001/api/v1/security/login. Elegir Body, raw y JSON; usar el siguiente cuerpo:

```
{
  "email": "demo@reactlab.test",
  "password": "React123!"
}
```

Copiar el token de la respuesta. Para GET /api/v1/security/institutos, seleccionar Authorization, Bearer Token y pegarlo. Para POST en esa misma ruta, conservar la autorización y enviar:

```
{
  "name": "Instituto de prueba",
  "code": "TEST-01",
  "city": "Mazatlán",
  "status": "Activo"
}
```

Usar el ID devuelto para probar PUT y DELETE. La API devuelve 201 al crear y 204 al eliminar. Para regresar al modo navegador, cambiar VITE_DATA_MODE a local y reiniciar Vite. Ambos modos conservan conjuntos de datos separados; no existe sincronización automática.

11 Configurar y ejecutar la PWA

Una PWA incorpora un manifest con los datos de instalación y un service worker que permite servir recursos previamente guardados. Se configuró vite-plugin-pwa en vite.config.js para generar ambos durante la compilación.

1. Agregar VitePWA a plugins junto con react. El manifest define nombre, nombre corto, idioma, alcance, ruta inicial y modo de visualización standalone.

```
// Extracto de la configuración del manifest
manifest: {
  name: 'React Lab · T4 Actividad 2',
  short_name: 'React Lab',
  lang: 'es',
  start_url: '/',
  scope: '/',
  display: 'standalone',
  background_color: '#f6f7f4',
  theme_color: '#172c28'
}
```

2. Añadir los iconos de 192 y 512 píxeles en public y declararlos en icons. El proyecto incluye además icon.svg. Workbox precachea los archivos JS, CSS, HTML, SVG y PNG del resultado de compilación.

3. Crear PwaControls.jsx con useRegisterSW. El componente avisa cuando la app está disponible sin conexión y cuando existe una actualización. registerType: prompt permite que el usuario guarde antes de actualizar.

4. Para probar la compilación PWA, usar modo local y ejecutar estos comandos desde la raíz:

```
npm run build
npm run preview
```

Abrir la dirección Local de preview, normalmente http://127.0.0.1:4173/, y esperar el aviso de disponibilidad sin conexión. La carpeta dist contiene el resultado, incluido manifest.webmanifest y sw.js. Preview sirve esa carpeta localmente; no publica el proyecto.

Instalación y prueba sin conexión

En un navegador compatible, utilizar Instalar app o la opción de instalación del menú. La disponibilidad depende del navegador y sus criterios. Para comprobar el caché, abrir las herramientas de desarrollo, activar Offline en Network y recargar la aplicación después de su primera carga completa.

El modo local conserva sus catálogos en localStorage y puede operar sin Internet una vez almacenados los recursos. La vista ReqRes requiere Internet. El modo Express depende de alcanzar el servidor local y no encola cambios cuando la API está desconectada. No se cachean respuestas autenticadas de los catálogos.

Si se cambia .env.local para probar otra arquitectura en preview, volver a ejecutar npm run build: las variables quedan incorporadas en los archivos compilados.

12 Verificar y demostrar la actividad

Pruebas automatizadas ejecutadas

Desde la raíz se ejecutó npm test. Las cuatro pruebas pasaron. Posteriormente npm run build completó la compilación y generó los recursos PWA. La revisión visual interactiva de la aplicación quedó pendiente; los siguientes pasos sirven para comprobarla manualmente.

Prueba	Qué comprueba
API de extremo a extremo	Acceso rechazado sin token, login, alta, detalle, edición, validación, baja y cierre de sesión.
CRUD de usuarios	Identidad estable al editar, duplicados y eliminación.
Eliminación en cascada	Borra los hijos del instituto y conserva los datos de otras instituciones.
Validación de entradas	Rechaza hijos sin padre, precios negativos, URL no admitidas y periodos invertidos.

Recorrido para presentar la aplicación

1. Abrir Inicio e iniciar sesión con las credenciales demo. En Usuarios, crear un registro, consultarlo, editarlo y buscarlo. Probar los botones de paginación.

2. Recargar, iniciar sesión y comprobar que el registro permanece. Eliminar el registro de prueba desde su diálogo de confirmación.

3. Abrir Hooks de React y probar ambos contadores. Abrir Consulta de API y cambiar entre páginas con conexión a Internet.

4. En Institutos, seleccionar una institución, abrir Negocios y seleccionar uno. Crear un teléfono o domicilio y comprobar que corresponde a ese negocio.

5. Crear una carrera en Educación y un producto en Comercio. Cambiar al modo API siguiendo la sección 10 y demostrar las solicitudes HTTP y el archivo JSON.

Problemas frecuentes

Síntoma	Revisión
La terminal no reconoce npm	Verificar la instalación de Node.js y volver a abrir la terminal.
La API no responde	Ejecutar npm run api y comprobar el puerto 3001. Revisar .env.local y reiniciar Vite.
No aparecen datos anteriores	Verificar modo, navegador, dirección y puerto. LocalStorage y el archivo JSON son almacenes distintos.
No aparece la instalación PWA	Usar build y preview, esperar al service worker y revisar compatibilidad del navegador.
ReqRes muestra un error	Revisar Internet, esperar si hay límite de consultas y usar Reintentar.

13 Relación con las notas de clase

La siguiente relación utiliza la numeración de los nombres de los archivos enviados. Permite localizar cada tema en el proyecto y explicar qué se implementó durante la actividad.

Nota	Tema	Archivo o componente principal
4.6	Estado con useState	src/pages/HooksLab.jsx
4.7	Custom hook useCounter	src/hooks/useCounter.js
4.8	Estado con useReducer	src/security/auth/AuthContext.jsx
4.9	Consulta HTTP con Axios	src/hooks/useUsuarios.js
4.10	Paginación	RemoteUsers.jsx y CrudTable.jsx
4.11	Custom hook useUsuarios	src/hooks/useUsuarios.js
4.12	Formularios controlados	src/pages/Login.jsx y RecordForm
4.13	Custom hook para formularios	src/hooks/useForm.js
6.1	Vite React y JavaScript	package.json y vite.config.js
6.2	Aplicación principal	src/main.jsx y AppAllModules.jsx
6.3	Rutas anidadas	AppAllModules.jsx y ModulePage.jsx
6.4	AppBar y navegación	src/components/Layout.jsx
6.5	Estructura Redux	src/security/redux/
6.6	Pestañas de institutos y negocios	src/security/institutes/pages/Institutes.jsx

Adaptaciones de la implementación

Se corrigieron nombres e imports inconsistentes de los ejemplos y se organizaron componentes reutilizables. El proyecto utiliza Material UI para mantener una interfaz común. La nota 6.6 prepara las pestañas; las operaciones CRUD y la API Express local completan el alcance solicitado en la actividad.

Los fragmentos de este tutorial explican partes concretas. El código completo, incluidos imports y manejo de errores, se encuentra en los archivos indicados. README.md conserva las instrucciones de ejecución y los scripts del proyecto.

Referencias de apoyo

Material de clase: PDF 4.6 a 4.13 y 6.1 a 6.6 proporcionados para la actividad.

ReqRes API docs: https://reqres.in/docs
Vite PWA Guide: https://vite-pwa-org.netlify.app/guide/
Redux Toolkit configureStore: https://redux-toolkit.js.org/api/configurestore/